FDFED M2026 — Lab 1: React Migration Planning

Existing Application Analysis & Architecture Blueprint

Project: ResourceX | Architecture: NestJS REST + Multi-Page Vanilla JS | Submission: 1 Official PDF Report | Scope: Analysis & Planning (No Implementation)

Lab Purpose: Analyze the existing multi-page application (**ResourceX**) and identify the architectural components, dynamic behaviors, API integrations, and code smells that will be redesigned using React during this semester. Per lab instructions, React implementation is NOT required for this lab. This document serves as the formal baseline analysis and migration blueprint.

1. APPLICATION FLOW ANALYSIS (TASK 0)

ResourceX is an Enterprise Resource Management and Asset Lifecycle platform serving educational and corporate institutions. The existing frontend is structured as a multi-page web application (MPA) interfacing with a monolithic NestJS REST backend.

Application Structure & Entry Points

- **Entry Pages:** `frontend/index.html` (Product showcase and public marketing landing page) and `frontend/pages/login.html` (Multi-step credential and tenant organization entry point).
- **Core Role-Based Dashboards:**
  - `requestor.html` — Resource requests, requisition tracking, active personal inventory, and procurement submissions.
  - `department_head.html` — Review, budgetary approval, and rejection of resource and procurement requests.
  - `staff.html` — Asset inventory maintenance, barcode serial allocation, return receiving, and threshold monitoring.
  - `registrar.html` — Asset registration, barcode tag verification, and physical audit control.
  - `admin.html` — Organization-level user provisioning, role assignments, department routing, and audit logs.
  - `owner.html` & `employee.html` — SaaS multi-tenant governance, tier subscriptions, and customer support.

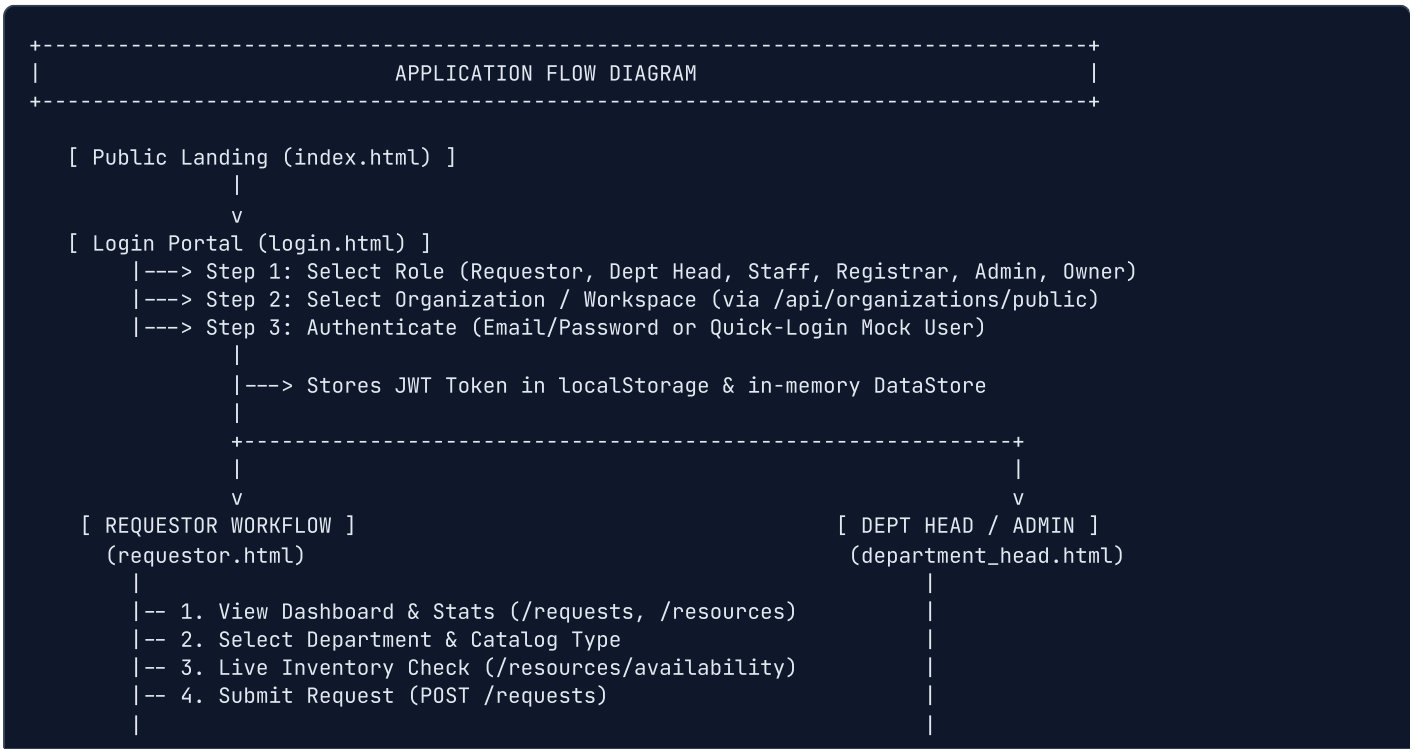
Core User Workflows

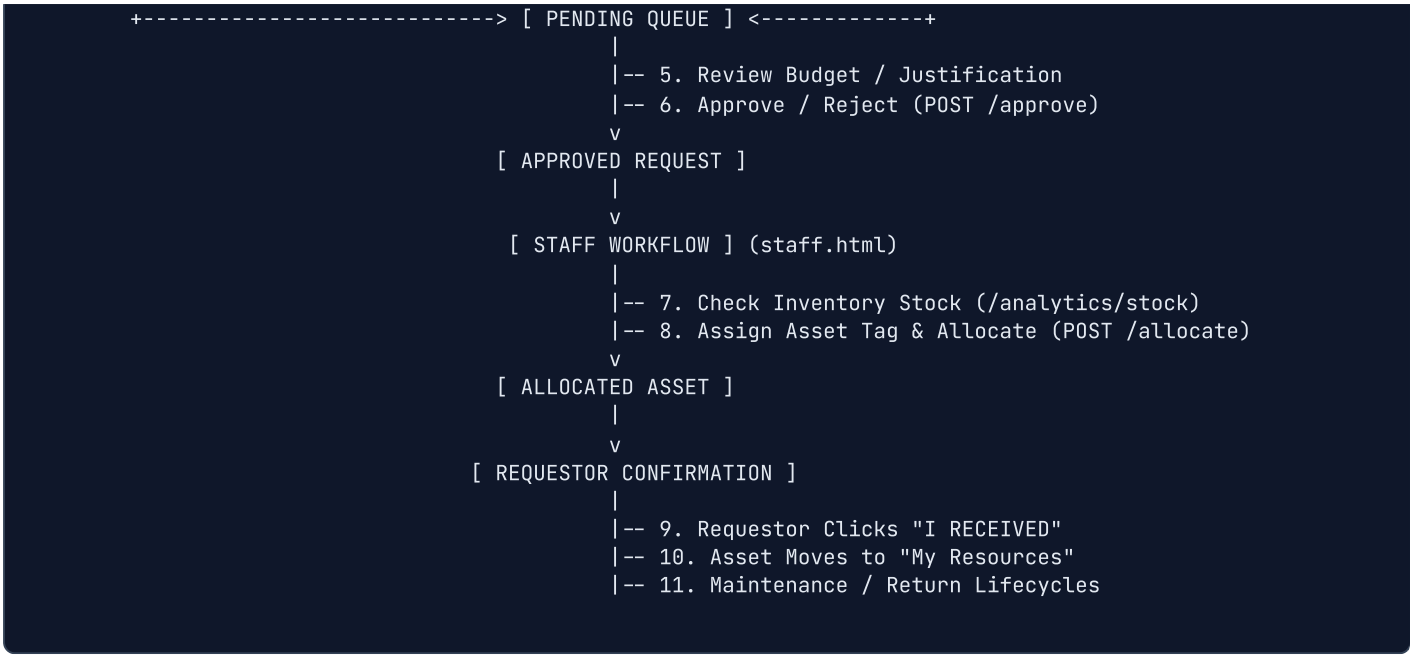
1. **Authentication & Multi-Tenant Routing:** User selects role → selects tenant organization → enters credentials → receives JWT bearer token → gets redirected to specific role dashboard.
2. **Resource Allocation Lifecycle:** Requestor submits item requisition → Dept Head approves/rejects → Staff physically checks out and allocates serial item → Requestor confirms physical receipt ("I RECEIVED").
3. **Procurement Requisition Pipeline:** For out-of-stock items, Requestor/Staff uploads specification document (drag-and-drop file upload) → Dept Head authorizes budget → Registrar logs barcode → Staff processes PO and uploads supplier invoice → Item is enrolled into active inventory.
4. **Maintenance & Returns Workflow:** Defective items are flagged by Requestor/Staff → asset moves to "In Maintenance" state → logged in maintenance history → returned to active inventory pool once repaired.

Data Ingestion & Presentation Touchpoints

Data Entry: Multi-step login wizard, resource request creation form, procurement file drag-and-drop upload zone, stock threshold inputs, and maintenance resolution forms.
Data Presentation: KPI stat metric cards, tabular lists with status badge pills, asset preview modals, unread notification dropdown badges, and toast alerts.
Backend Integration: Mediated via `frontend/js/store.js` which performs asynchronous REST API calls over HTTP with JWT bearer authentication headers.

Application Flow Diagram




+----------------------------> [PENDING QUEUE] <-------------+
|
|-- 5. Review Budget / Justification
|-- 6. Approve / Reject (POST /approve)
v
[APPROVED REQUEST]
|
v
[STAFF WORKFLOW] (staff.html)
|
|-- 7. Check Inventory Stock (/analytics/stock)
|-- 8. Assign Asset Tag & Allocate (POST /allocate)
v
[ALLOCATED ASSET]
|
v
[REQUESTOR CONFIRMATION]
|
|-- 9. Requestor Clicks "I RECEIVED"
|-- 10. Asset Moves to "My Resources"
|-- 11. Maintenance / Return Lifecycles

2. REPEATED UI ANALYSIS (TASK 1)

An inspection across all application pages reveals numerous visual and structural elements that are duplicated verbatim across multiple HTML templates. These present prime candidates for encapsulation into reusable React components.

UI ELEMENT	PAGES / LOCATION	HOW IS IT REPEATED?	WHY COULD REUSE BE USEFUL?
Example: Footer	All pages	Same footer on every page	One change can be reflected everywhere
Global Top Header	All 8 role dashboards (`requestor.html` , `staff.html` , etc.)	Rendered via custom Web Component `<app-header>` ; replicates search bar, notification bell dropdown, and avatar menu.	Unified state synchronization for unread notification count, active user profile, and global logout handling without redundant code.
Navigation Sidebar	All dashboard pages	Rendered via `<app-sidebar>` with duplicated CSS rules and copy-pasted menu item structures.	Allows single-point configuration of role-based navigation links, active tab highlighting, and branding.
KPI Metric Cards	`requestor.html` , `staff.html` , `dept_head.html` , `admin.html`	4-column grid of `.stat-card` containing label, numeric counter, icon, and left accent border.	A reusable `<StatCard>` component reduces duplicated boilerplate and standardizes responsive styling across all roles.
Data Table & Striping	Requests, Inventory, Procurement, Maintenance views across all pages	Standard `<table class="table">` with repeated `<thead>` styling, zebra striping, and empty-state rows.	Encapsulating a generic `<DataTable>` enables universal sorting, pagination, and zero-state messaging across all data grids.
Status Badge Pills	All request tables, procurement cards, asset lists	Inline HTML spans like `<span class="badge pending">` with identical color mapping logic duplicated in JS strings.	A single `<StatusBadge status={status} />` guarantees consistent color semantics (e.g., green for Allocated, yellow for Pending).
File Upload Dropzone	`requestor.html` (procurement), `staff.html` (invoice upload)	Dashed boundary box with drag-and-drop listeners, file preview card, file size formatting, and delete action.	Eliminates 120+ lines of duplicated drag-and-drop boilerplate into a reusable `<FileUploadZone>` with integrated validation.
Modal Dialogs	`staff.html` , `dept_head.html` , `admin.html`	Duplicated backdrop overlay, modal header with close button, body content, and action button footers.	Prevents DOM clutter; a reusable `<Modal>` component with React portal manages accessibility and focus trapping cleanly.
Toast Alert System	Global application scope (`store.js`)	Imperatively created DOM `<div class="toast">` appended directly to `document.body` .	A centralized `<ToastContainer>` driven by React state cleanly manages auto-dismiss timers, queueing, and animations.

React Connection: Identifying these repeated UI patterns confirms that over 65% of the existing HTML across dashboard pages consists of identical layout and data presentation structures. Migrating to React components will drastically reduce code volume and eliminate UI drift.

3. DYNAMIC BEHAVIOUR ANALYSIS (TASK 2)

Dynamic behavior occurs whenever user interaction triggers a state change, DOM transformation, or network operation. The following table catalogs the critical dynamic interactions observed across the application:

PAGE / FEATURE	USER ACTION	WHAT CHANGES IN THE UI / APPLICATION?
Example: Products	Enter search text	Product list changes
Login Wizard	User clicks a role button (e.g., "Requestor")	Role selection container hides (`.step-container` removes `active`); organization selection dropdown step displays with smooth fade-in animation.
Login Wizard	User clicks " ⚡ Quick Login"	Asynchronously fetches pre-seeded mock credentials from `/api/users/mock` , auto-populates email and password fields, and automatically triggers submission.
Login Validation	User submits empty or invalid email	Hides error state, displays inline `#emailError` warning text in red, borders input in `#ef4444` , and focuses the field.
Sidebar Navigation	User clicks a navigation item (e.g., "My Requests")	Removes `.active` from all nav items, marks clicked item active, hides all `.view-section` containers, and reveals target section without full-page reload.
Request Resource Form	User selects a Department from dropdown	Triggers `updateResourceDropdown()` ; reads catalog for that department and dynamically populates the Resource Type `<select>` with available items.
Request Resource Form	User selects a Resource Type	Fires asynchronous request to `/resources/availability` ; dynamically renders inventory count text: displays in green if count > 0, or red warning if count is 0.
Procurement Upload	User drops a file onto upload zone	Validates MIME type (PDF/DOCX/PNG/JPG) and 5MB limit; reads base64 data via `FileReader` ; swaps dashed dropzone with a green preview badge showing file name, size, and remove button.
My Requests Table	User selects a status filter (e.g., "Pending")	Filters in-memory request array by status; wipes `<tbody>` and re-renders matching table rows instantly with appropriate badge styles.
My Requests Table	User clicks "I RECEIVED" button on Allocated item	Prompts user confirmation dialog; dispatches API status update; updates row badge to "Completed"; transfers item to "My Resources" table; updates KPI counts.
Staff Inventory	User edits threshold and clicks Save	Updates local stock threshold array; persists to backend; dynamically updates inventory status badge from "In Stock" to "Low Stock Warning" if quantity ≤ threshold.
Top Header	User clicks notification bell icon	Toggles `.show` class on dropdown menu; renders unread notifications; marks notifications as read; clears red badge counter.

React Connection: In vanilla JS, all these state transitions require manual imperative DOM queries and class list manipulations. In React, these represent standard declarative state variables (`selectedRole` , `filterStatus` , `availableCount` , `uploadedFile`) bound directly to JSX elements.

4. EXISTING JAVASCRIPT ANALYSIS (TASK 3)

A code audit of `frontend/js/` reveals extensive reliance on manual DOM selection, imperative node creation, and string-based HTML injection. The table below documents representative examples from the codebase:

JAVASCRIPT CODE / PATTERN	WHAT DOES IT DO?	PAGE / FEATURE	WHAT UI DOES IT AFFECT?
Example: getElementById()	Finds a DOM element	Cart	Cart count
`document.getElementById('dash-total-req')`	Finds a specific DOM element by ID to update its text	`requestor.js` (lines 394, 399)	Total Requests stat card counter number on dashboard
`document.querySelector('.user-name')`	Queries the first element matching class selector	`requestor.js` (line 22) / `staff.js`	Displays logged-in user's full name in the sidebar widget
`document.querySelectorAll('.view-section')`	Returns a static NodeList of all view containers	`requestor.js` (line 162) / `staff.js`	Iterates over sections to strip `.active` class during tab transitions
`addEventListener('click', ...)`	Attaches an imperative click listener callback	`requestor.js` (lines 148-158)	Sidebar navigation links to toggle view states without page refresh
`zone.addEventListener('drop', ...)`	Listens for file drop events and extracts file object	`requestor.js` (lines 45-50)	Procurement drag-and-drop file upload container
`tbody.innerHTML = ''`	Wipes all child nodes inside a table body	`requestor.js` (lines 407, 457, 496)	Clears the request/resource table before rebuilding rows
`tr.innerHTML = `<td>${req.id}</td>...``	Injects raw HTML template strings into the DOM	`requestor.js` (lines 411, 480, 516)	Dynamically constructs table rows, badges, and action buttons
`element.classList.add('active')`	Adds a CSS class to an element's classList	`login.html` (line 213) / `staff.js`	Controls visibility and fade-in animations of wizard steps and views
`document.createElement('option')`	Creates a new DOM option node programmatically	`requestor.js` (lines 181, 216)	Populates department and resource type select dropdown lists
`zone.style.display = 'none'`	Directly modifies element inline CSS display style	`requestor.js` (line 100)	Hides upload dropzone and displays file preview box

React Connection: This analysis demonstrates the overhead of imperative DOM manipulation. In React, JSX replaces `innerHTML` string concatenation, declarative conditional rendering replaces manual `classList` and `style.display` mutations, and synthetic event handlers (`onClick` , `onDrop`) replace manual `addEventListener` bindings.

5. NESTJS API ANALYSIS (TASK 4)

The ResourceX backend is implemented as a modular NestJS application exposing a REST API. The frontend communicates with these endpoints using the native `fetch()` API wrapped inside `Store.api()` in `frontend/js/store.js`.

Architecture Representation

```
Existing Architecture:

   Frontend (Vanilla HTML/CSS/JS + Web Components)
          |
          v  REST API (HTTP / JSON / JWT Bearer Authorization)
       NestJS (Controllers, Guards, Services, DTOs, Swagger Docs)
          |
          v  Data Layer (Prisma ORM / File Repository)
       Database (PostgreSQL / SQLite Storage Engine)
```

REST API Endpoints Inventory

FEATURE AREA	HTTP METHOD	API ENDPOINT	PURPOSE & DESCRIPTION
Example: Login	`POST`	`/auth/login`	Authenticate user
Authentication	`POST`	`/api/auth/login`	Validates credentials, returns JWT bearer token and user role profile.
Organizations	`GET`	`/api/organizations/public`	Retrieves public organization list for tenant selection on login.
Mock Auth	`GET`	`/api/users/mock?role=&orgId=`	Provides seed test credentials for 1-click quick-login testing.
Resource Requests	`GET`	`/api/requests`	Lists requests scoped to the authenticated user's role and department.
Resource Requests	`POST`	`/api/requests`	Submits a new resource requisition with item type, qty, and reason.
Request Review	`POST`	`/api/requests/:id/approve`	Authorizes a pending resource request (Dept Head / Admin).
Request Review	`POST`	`/api/requests/:id/reject`	Rejects a resource request with recorded feedback reason.
Request Allocation	`POST`	`/api/requests/:id/allocate`	Assigns specific physical inventory barcode/asset to approved request.
Inventory Resources	`GET`	`/api/resources`	Retrieves all physical inventory assets, serials, and current status.
Live Availability	`GET`	`/api/resources/availability`	Queries real-time available stock count for a given department and item.
Resource Catalog	`GET`	`/api/resources/catalog`	Returns configured resource categories mapped per department.
Procurements	`GET`	`/api/procurements`	Fetches procurement requisitions and multi-stage lifecycle statuses.

FEATURE AREA	HTTP METHOD	API ENDPOINT	PURPOSE & DESCRIPTION
Procurements	`POST`	`/api/procurements`	Submits procurement requisition along with specification attachment data.
Maintenance Logs	`GET`	`/api/maintenance/history`	Lists maintenance tickets, service costs, and asset repair history.
Stock Analytics	`GET`	`/api/analytics/stock`	Queries inventory stock levels against alert thresholds.
Notifications	`GET`	`/api/notifications`	Retrieves system alerts, status updates, and actionable notifications.

Lab Constraint Adherence: Per the course instructions, the existing NestJS backend and its endpoints will be preserved without modification. The upcoming React migration will exclusively replace the presentation layer while consuming these exact endpoints.

6. PROBLEMS IN THE EXISTING FRONTEND (TASK 5)

A rigorous critical evaluation of the existing codebase identified the following 7 authentic engineering deficiencies:

CURRENT ISSUE	WHERE DOES IT OCCUR?	WHY IS IT A PROBLEM? (TECHNICAL IMPACT)
Example: Repeated product card	Products	Changes may need to be made repeatedly
Unsafe `innerHTML` String Concatenation	`requestor.js`, `staff.js`, `admin.js`	HTML rows are constructed via unescaped string interpolation (e.g., `tr.innerHTML = \`<td>${req.reason}</td>\``). If a user submits malicious HTML/JS in the request justification, it executes directly in the DOM, introducing Cross-Site Scripting (XSS) vulnerabilities.
Destructive Full-Table Re-rendering	All `renderX()` functions across all pages	Every minor state change calls `tbody.innerHTML = ''` and reconstructs every single DOM node from scratch. This destroys active text selection, resets scroll position, breaks keyboard focus accessibility, and triggers severe browser reflow and layout thrashing.
Monolithic JavaScript File Bloat	`staff.js` (1,253 lines), `admin.js` (950 lines), `requestor.js` (675 lines)	Violates the Single Responsibility Principle. DOM querying, event handling, data transformation, validation, and REST API dispatching are tightly coupled inside a single colossal object. Maintenance is brittle and refactoring carries high risk of unintended regressions.
Duplicated Boilerplate Across Multiple Files	`bindNav`, `switchView`, and modal markup across 8 HTML/JS files	Tab-switching algorithms and modal skeletons are copy-pasted verbatim across all dashboard files. If navigation semantics or modal animations need updating, the developer must manually edit 8 different files, leading to inevitable UI inconsistencies.
Monolithic Unthrottled Sync Loop	`store.js` (`sync()` method lines 163–190)	Whenever any action completes (e.g., submitting a single request), `Store.sync()` dispatches 10 parallel HTTP requests simultaneously to re-fetch *every single backend collection* (users, departments, requests, catalog, notifications, etc.). This causes severe network overhead and race conditions.
Global Window Scope Leakage	Inline HTML event attributes (`onclick="app.switchView(...)"`)	Interactive buttons rely on global objects (`window.app`, `window.staffApp`). This makes modular testing impossible, breaks if script loading order shifts, and prevents proper garbage collection of attached handlers.
Lack of Form State Isolation	Request creation and procurement forms	Form input values are queried directly from the DOM on submit using `getElementById().value`. There is no reactive form state, making field-level validation, live character counts, dynamic error messaging, and button-disabling difficult and cumbersome.

7. REACT CONVERSION OPPORTUNITIES

Migrating from vanilla multi-page scripts to React 18+ directly resolves the technical debt documented above through the following modern frontend paradigms:

- **Declarative UI & Virtual DOM Reconciliation:** Replacing manual `innerHTML` string concatenation with declarative JSX syntax. The Virtual DOM computes efficient diffs, updating only mutated nodes rather than wiping and rebuilding entire tables. Built-in JSX escaping eliminates XSS vulnerabilities.
- **Component-Based Modularity:** Decomposing monolithic pages into atomic, reusable components:
  - *Layout:* `<AppLayout>`, `<Sidebar>`, `<Header>`.
  - *Data Presentation:* `<StatCard>`, `<DataTable>`, `<StatusBadge>`.
  - *Interactive Controls:* `<FileUploadZone>`, `<FilterBar>`, `<Modal>`.
- **Reactive State Management via React Hooks:**
  - `useState` / `useReducer` for isolated component states (e.g., active tab, search filter, modal visibility).
  - `useEffect` for predictable lifecycle data fetching with cleanup.
  - `useMemo` and `useCallback` for instant, optimized client-side filtering without redundant re-renders.
- **Context API for Global State (`AuthContext`):** Eliminates fragile `localStorage` polling and replaces the monolithic 10-call `Store.sync()` loop with granular, feature-specific data hooks (`useRequests`, `useCatalog`).
- **Single-Page Application (SPA) Routing (`react-router-dom`):** Replaces disjointed HTML page reloads with instantaneous client-side transitions, preserving layout state, user preferences, and browser history.

8. FIRST REACT CONVERSION TARGET

Selected Feature Target: The Requestor Portal & Resource Request Management Workflow (`frontend/pages/requestor.html` + `frontend/js/pages/requestor.js`).

Strategic Justification for Selection

1. **Core Business Engine of the Platform:** The Requestor role represents the primary consumer persona in ResourceX. All downstream activities across Department Heads, Staff inventory allocation, and Registrar asset tagging originate directly from resource requisitions submitted by Requestors. Modernizing this flow delivers immediate, high-impact value.
2. **Ideal Architectural Complexity:** Unlike `staff.js` or `admin.js` (which contain over 1,200 lines spanning 15 sub-modules), the Requestor portal is self-contained yet technically rich. It encompasses all core React patterns:
   - Interactive multi-attribute form input with dynamic cascading dropdowns.
   - Asynchronous live API inventory verification.
   - Drag-and-drop file upload with FileReader binary handling.
   - Tabular data rendering with dynamic status filtering and action dispatching.
3. **High Dynamic Interaction Density:** The Requestor workflow currently suffers most from manual DOM manipulation (rebuilding tables via string templates, manual file preview toggling, and imperative tab switching). Converting this page demonstrates the clearest contrast in code quality, maintainability, and user experience.
4. **Foundation Blueprint for Future Labs:** The React components developed for this target (`<StatCard>` , `<DataTable>` , `<StatusBadge>` , `<FileUploadZone>`) will serve as direct, drop-in building blocks for subsequent Staff and Dept Head lab conversions.

9. INITIAL REACT CONVERSION SKETCH

The following structural design outlines the component hierarchy, state flow, and interface architecture for the initial React conversion of the Requestor Portal.

Component Hierarchy Tree

```
<App>
  └── <AuthProvider>
        └── <RequestorPortal>
              ├── <AppSidebar activeTab={activeTab} onTabChange={setActiveTab} />
              └── <MainContentWrapper>
                    ├── <AppHeader user={currentUser} />
                    └── <main className="content-area">
                          {/* 1. Dashboard Overview Tab */}
                          {activeTab === 'dashboard' && (
                            <DashboardView>
                              <StatsGrid stats={summaryStats} />
                              <div className="overview-grid">
                                <RecentRequestsCard requests={requests.slice(0, 5)} />
                                <MyResourcesCard resources={resources.slice(0, 5)} />
                              </div>
                            </DashboardView>
                          )}

                          {/* 2. New Request Tab */}
                          {activeTab === 'new-request' && (
                            <RequestForm
                              departments={departments}
                              catalog={resourceCatalog}
                              onSubmit={handleCreateRequest}
                            />
                          )}

                          {/* 3. My Requests Tracking Tab */}
                          {activeTab === 'my-requests' && (
                            <RequestsListView
                              requests={requests}
                              onConfirmReceipt={handleConfirmReceipt}
                            />
                          )}

                          {/* 4. Procurement Tab */}
                          {activeTab === 'procurement' && (
                            <ProcurementView
                              procurements={procurements}
                              onSubmitProcurement={handleCreateProcurement}
                            >
                              <FileUploadZone onFileChange={setProcSpecFile} />
                            </ProcurementView>
                          )}
                    </main>
```

Component Specifications & State Distribution

COMPONENT NAME	LOCAL STATE (USESTATE)	PROPS RECEIVED	EVENT HANDLERS
`<RequestorPortal>`	`activeTab`, `requests[]`, `resources[]`, `loading`	None (Top-level feature container)	`refreshData()`, `handleCreateRequest()`
`<StatsGrid>`	None (Pure presentational)	`stats: { total, pending, allocated, maint }`	None
`<RequestForm>`	`dept`, `resourceType`, `qty`, `reason`,	`departments[]`, `catalog[]`	`onDeptChange()`, `onTypeChange()`,

COMPONENT NAME	LOCAL STATE (USESTATE)	PROPS RECEIVED	EVENT HANDLERS
	`availableStock`, `isSubmitting`		`handleSubmit()`
`<RequestsListView>`	`statusFilter`, `searchTerm`	`requests[]`	`onFilterChange()`, `onConfirmReceipt(id)`
`<FileUploadZone>`	`isDragOver`, `errorMsg`, `filePreview`	`accept`, `maxSizeMB`, `onFileChange`	`handleDrop()`, `handleSelect()`, `handleRemove()`
`<StatusBadge>`	None (Pure presentational)	`status: string`	None

Visual UI Layout Wireframe

```
+-------------------------------------------------------------------------------------------------------+ |
ResourceX Header [ Search resources... ] [🔔 3] [ User: Ravi Chandra (Req) v] | +------------------+------
-------------------------------------------------------------------+ | [R] ResourceX | Welcome back,
Ravi Chandra! | | | Overview of your departmental resource requests and active inventory. | | • Dashboard
| | | • Request Item | +----------------+ +----------------+ +----------------+ +----------------+ | | •
My Requests | | Total Requests | | Pending Appr. | | My Resources | | In Maintenance | | | • My Resources
| | 12 | | 02 | | 05 | | 01 | | | • Procurement | +----------------+ +----------------+ +----------------+
+----------------+ | | | | | Recent Resource Requests | | | +----------+--------------------+-----+-----
---------------+------------+ | | | | Req ID | Resource Type | Qty | Status | Action | | | | +----------+-
-------------------+-----+--------------------+------------+ | | | REQ-104 | Developer Laptop | 01 |
[Allocated (Green)]| [RECEIVED] | | | | | REQ-108 | 4K Display Monitor | 02 | [Pending (Yellow)] |
[Details] | | | | | REQ-099 | Office Ergonomic | 01 | [Rejected (Red)] | [Reason] | | | | +----------+----
---------------+-----+--------------------+------------+ | +------------------+--------------------------
-----------------------------------------------------+
```

Component Skeleton & Hook Integration Pseudo-Code

```
// Target Component Sketch: RequestForm.jsx
import React, { useState, useEffect } from 'react';
import { useAuth } from '../context/AuthContext';
import { fetchAvailability, submitRequest } from '../services/api';

export function RequestForm({ departments, catalog, onRequestSubmitted }) {
  const { user } = useAuth();
  const [formData, setFormData] = useState({ department: '', resourceType: '', quantity: 1, reason: '' });
  const [availableCount, setAvailableCount] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  // Derive catalog options dynamically based on selected department
  const resourceOptions = catalog.find(c => c.department === formData.department)?.resourceTypes || [];

  // Reactive effect: fetch live inventory count when type or department changes
  useEffect(() => {
    if (!formData.department || !formData.resourceType) {
      setAvailableCount(null);
      return;
    }
    let isMounted = true;
    fetchAvailability(formData.department, formData.resourceType).then(res => {
      if (isMounted) setAvailableCount(res.availableCount);
    });
    return () => { isMounted = false; };
  }, [formData.department, formData.resourceType]);

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);
    setError(null);
    try {
```

```
      await submitRequest({ ...formData, requesterId: user.id });
      onRequestSubmitted();
    } catch (err) {
      setError(err.message || 'Failed to submit request');
    } finally {
      setLoading(false);
    }
  };

  return (
    <form className="card form-container" onSubmit={handleSubmit}>
      <h3>Request Resource</h3>
      {error && <div className="error-banner">{error}</div>}

      <label>Department</label>
      <select value={formData.department} onChange={e => setFormData({ ...formData, department: e.target.value
        <option value="" disabled>Select Department</option>
        {departments.map(d => <option key={d.name} value={d.name}>{d.name}</option>)}
      </select>

      <label>Resource Type</label>
      <select value={formData.resourceType} onChange={e => setFormData({ ...formData, resourceType: e.target.v
        <option value="" disabled>Select Resource Type</option>
        {resourceOptions.map(t => <option key={t} value={t}>{t}</option>)}
      </select>

      {availableCount !== null && (
        <p style={{ color: availableCount > 0 ? '#16a34a' : '#dc2626', fontWeight: 600 }}>
          Available in Inventory: {availableCount} units
        </p>
      )}

      <label>Quantity</label>
      <input type="number" min="1" value={formData.quantity} onChange={e => setFormData({ ...formData, quantit

      <label>Justification / Purpose</label>
      <textarea value={formData.reason} onChange={e => setFormData({ ...formData, reason: e.target.value })} r

      <button type="submit" className="btn-primary" disabled={loading}>
        {loading ? 'Submitting...' : 'Submit Resource Request'}
      </button>
    </form>
  );
}
```